

Manual Avanzado React para Aprobar el Examen

1. Fundamentos

- React = librería para interfaces dinámicas.
- Componente = función que devuelve JSX.
- JSX = sintaxis tipo HTML + JavaScript.
- `{}` dentro de JSX = ejecutar JavaScript.
- Un componente debe devolver un único elemento raíz.
- Exportar/importar componentes con `export default` e `import`.

2. Arquitectura típica de examen

```
App
├── SpotiMain
│   ├── ListadoCanciones
│   ├── FiltradoCanciones
│   └── MasPopular
```

- Componente padre: carga datos. - Componentes hijos: muestran o transforman datos.

3. Props

Props = pasar datos de padre a hijo.

Padre:

```
<ListadoCanciones canciones={canciones} />
```

Hijo:

```
const ListadoCanciones = ({ canciones }) => {
```

Reglas:

- Props no se modifican.
- Se pueden pasar strings, números, arrays, objetos.

4. useState

Guardar datos dinámicos.

```
const [canciones, setCanciones] = useState([]);
```

- `canciones` = valor actual
- `setCanciones()` = cambiar valor
- Si cambia el estado, React renderiza otra vez.

Nunca:

```
canciones = datos;
```

Siempre:

```
setCanciones(datos);
```

5. useEffect

Ejecutar código al cargar.

```
useEffect(() => {  
  cargarCanciones();  
}, []);
```

- `[]` = solo una vez.
- Perfecto para cargar JSON.

6. Fetch + async/await

```
const cargarCanciones = async () => {  
  const respuesta = await fetch('/json/Spotify.json');  
  const datos = await respuesta.json();  
  setCanciones(datos);  
};
```

Conceptos:

- `async` = permite usar await
- `await` = espera
- `fetch` = pide datos
- `.json()` = convierte respuesta a JS

7. map()

Recorrer arrays y pintar listas.

```
{canciones.map((cancion) => (
  <tr key={cancion.track_id}>
    <td>{cancion.track_name}</td>
  </tr>
))}
```

Reglas:

- Siempre usar `key`
- `map()` devuelve un elemento por cada item

8. filter()

Filtrar arrays.

```
const cancionesFiltradas = canciones.filter((cancion) =>
  cancion.track_artist.toLowerCase().includes(artista.toLowerCase())
);
```

Conceptos:

- `includes()` = contiene texto
- `toLowerCase()` = ignora mayúsculas

9. reduce()

Obtener un único resultado.

```
const masPopular = canciones.reduce((max, cancion) =>
  cancion.track_popularity > max.track_popularity ? cancion : max
);
```

Antes:

```
if (canciones.length === 0) {
  return <p>Cargando...</p>;
}
```

10. Eventos

onClick

```
<button onClick={() => hacerAlgo()}>
```

onChange

```
onChange={(e) => setArtista(e.target.value)}
```

Conceptos:

- `e` = evento
- `e.target` = elemento
- `e.target.value` = valor escrito

11. Formularios

```
<input  
  type='text'  
  value={artista}  
  onChange={(e) => setArtista(e.target.value)}  
>
```

Input controlado = React controla el valor.

12. Renderizado condicional

Con if:

```
if (canciones.length === 0) {  
  return <p>Cargando...</p>;  
}
```

Con ternario:

```
condicion ? valor1 : valor2
```

13. Plantillas de examen

App.jsx

```
import SpotiMain from './components/SpotiMain';  
  
function App() {  
  return <SpotiMain />;  
}  
  
export default App;
```

SpotiMain.jsx

```
import { useEffect, useState } from 'react';
```

Cargar JSON + pasar props.

ListadoCanciones

Usa `map()`.

FiltradoCanciones

Usa `useState + filter + onChange`.

MasPopular

Usa `reduce()`.

14. JSON real del examen

```
track_id
track_name
track_artist
track_popularity
track_album_name
duration_ms
```

NO usar nombres inventados.

15. Errores típicos

- Olvidar imports
- Usar nombres JSON incorrectos
- Olvidar `key`
- `class` en vez de `className`
- No cerrar `<input />`
- `reduce()` con array vacío
- Ejecutar función directamente en evento

Mal:

```
onClick={hacerAlgo()}
```

Bien:

```
onClick={() => hacerAlgo() }
```

16. Orden exacto para resolver examen

1. Crear SpotiMain
2. Mostrar SpotiMain en App
3. Cargar JSON con fetch
4. `console.log(datos)`
5. Crear ListadoCanciones
6. Crear FiltradoCanciones
7. Crear MasPopular
8. Revisar nombres JSON

17. Chuleta final

```
useState = guardar datos  
useEffect = ejecutar al cargar  
props = pasar datos  
map = recorrer lista  
filter = filtrar  
reduce = obtener uno  
fetch = cargar JSON  
await = esperar  
onChange = detectar escritura  
e.target.value = texto input
```